MEDIXAI An Explainable Classical AI Expert System for Symptom Rule Analysis

Nishanth Elango Rajan, Naveen Raj P

***Abstract*—MEDIXAI is a browser-based expert system that demonstrates classical artificial intelligence through transparent evaluation of symptom rules. The system combines a manually encoded knowledge base, weighted overlap ranking, required-premise checks, and an interactive explanation interface. The implemented knowledge base contains 38 disease rules, 104 distinct symptom identifiers, and eight categories. Each rule separates weighted supporting symptoms from premises required for an academic implication to fire. Present, explicitly absent, and unknown symptom states support individual follow-up questions without interpreting every unselected symptom as negative evidence. The score normalizes matched symptom weight and applies a fixed 0.60 multiplier whenever a required premise is missing; it is an overlap measure rather than a probability of disease. Verification of the supplied JavaScript used 175 assertions covering rule structure, complete profiles, premise deletion, input contradictions, state transitions, and rendered warnings. All assertions matched the characterized behavior, including two limitations: blocked candidates remain rankable, and contradiction handling takes precedence over dynamic warning rendering. The contribution is an inspectable educational implementation and a precise account of its reasoning semantics. The study establishes software behavior only; it provides no evidence of clinical diagnostic accuracy, validated triage, or treatment effectiveness.**

***Index Terms*—**Classical artificial intelligence, expert systems, knowledge representation, rule-based reasoning, explainability, symptom analysis.

I. INTRODUCTION

A symptom-based expert system provides a concrete setting for studying knowledge representation and inference. A learner can enter facts, inspect a rule antecedent, and compare a conclusion with the evidence that supports it. The difficulty is that a ranked disease label can appear more authoritative than the underlying symbolic model warrants. A useful educational implementation must therefore expose the difference between evidence overlap, satisfied premises, and a medically established diagnosis.

The MEDIXAI project specification describes an academic disease expert system based on classical AI concepts and supplies Python examples [1]. The accompanying HTML implementation [2] ports the expanded rule dictionary into a self-contained browser interface. This article analyzes the HTML implementation as the implemented artifact and uses the PDF to establish its design lineage. The application offers symptom collection, rule inspection, ranked candidates, and interactive checks of missing premises.

The work addresses three technical questions. First, how can a small symbolic knowledge base expose both partial evidence and complete antecedent satisfaction? Second, how should an interface distinguish unknown symptoms from explicitly negative answers? Third, what can deterministic verification establish when patient records and clinical reference labels are unavailable? These questions define the scope of the study; no machine-learning model or statistical diagnosis estimator is introduced.

The contributions are an explicit formalization of the implemented scoring and premise semantics, a description of the interaction between forward rule scanning and hypothesis-directed questioning, and a reproducible software characterization covering all 38 rules. The practical contribution lies in combining these mechanisms in an inspectable educational interface. The study does not claim a new inference calculus or priority over existing expert-system methods.

II. BACKGROUND AND RELATED WORK

A. Symbolic knowledge and medical expert systems

Knowledge-based agents represent domain statements explicitly and use inference to derive consequences from available facts [3]. MEDIXAI adopts a restricted propositional form: a conjunction of named symptom tokens supports a named disease rule. Its labels function as ground symbols for one interaction. Although the project brief mentions first-order logic, the implemented JavaScript does not contain quantified patient predicates, variable substitution, or unification.

MYCIN is an established reference for rule-based medical reasoning. Shortliffe and Buchanan studied inexact reasoning and its numerical representation within that system [4]. The subsequent MYCIN volume discusses knowledge engineering, explanations, uncertainty, and evaluation [5]. These works motivate separating a rule explanation from a claim of clinical correctness. MEDIXAI's weighted overlap formula is a simpler project heuristic; it is not MYCIN certainty-factor propagation or a Bayesian posterior.

B. Inference scope and consistency

The source describes forward chaining, backward chaining, and resolution-style checking [1]. In the browser artifact, forward reasoning is a single scan of independent symptom-to-disease rules. Backward reasoning inspects the selected rule's required premises and asks for missing facts. Neither operation recursively expands a network of intermediate goals. Formal resolution operates over logical clauses and can support refutation procedures [6]. The application instead checks three declared pairs of incompatible input tokens. This narrower description avoids attributing theorem-proving capabilities to a direct consistency test.

Health-oriented AI also requires attention to responsible use. WHO guidance places ethics and human rights within design, deployment, and governance [7]. MEDIXAI's educational interface labels its rules as unvalidated and avoids prescribing medication. These interface choices limit the intended use but do not establish clinical safety. The relevant contribution is transparency about a small symbolic model and the behavior that has actually been verified.

III. SYSTEM ARCHITECTURE AND KNOWLEDGE REPRESENTATION

A. Implementation boundary

The supplied application is a single HTML document with embedded CSS and JavaScript [2]. Its knowledge base and reasoning routines execute in the browser. Source inspection reveals no network request, remote inference service, external library dependency, or persistence API. The session maintains facts and answers in JavaScript sets. This supports local educational use and easy inspection; it does not constitute a security or privacy certification.

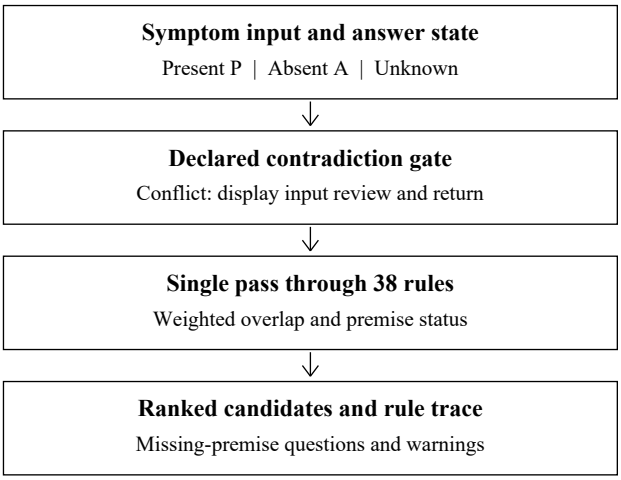


Fig. 1. MEDIXAI reasoning workflow. The contradiction gate precedes ranking and result rendering; the always-visible educational notice is outside this dynamic path.

Figure 1 shows the reasoning path. The user first selects symptom facts from a searchable vocabulary. A category filter restricts which symptom checkboxes are shown; it does not restrict which disease rules are evaluated. The consistency gate checks the present fact set. Valid inputs are passed to the scoring routine, and the results interface displays all rules with at least one matching symptom. Selecting a candidate opens its explanation and premise questions.

B. Knowledge base structure

Each disease entry stores a category, severity label, urgency flag, description, weighted symptom map, required-symptom list, treatment text, and precaution text. Table I summarizes the implemented category distribution. There are 228 weighted disease-symptom links and 79 required-premise links. Every rule has six weighted symptoms and at least one required premise. All required premises occur in the corresponding weighted map. Nine entries carry a source urgency flag.

TABLE I IMPLEMENTED KNOWLEDGE BASE

Category	Rules
Respiratory	8
Viral	6
Bacterial	5
Digestive	5
Skin	4
Neurological	3
Urinary	3
Miscellaneous	4
Total	38

The PDF labels the expanded implementation as a 40-disease version, but the supplied dictionary and browser artifact contain 38 entries [1], [2]. Accordingly, all reported counts use the implemented dictionary. The symptom weights and rule antecedents are manually specified project data. No clinical derivation, expert annotation procedure, or empirical calibration is documented for them. Category labels organize the interface and should not be treated as a validated clinical taxonomy.

C. Symptom states and academic implications

Let S denote the vocabulary, P the set of present facts, and A the set of explicitly absent facts. The interface maintains P and A as disjoint sets. A symptom outside their union is unknown. Unchecking a selected symptom returns it to unknown; an explicit No answer records absence. The token no_fever is itself an ordinary selectable token, separate from recording fever in A. There is no general negation-normalization layer relating these encodings.

$$v(s) = \begin{cases} \text{present} & s \in P \\ \text{absent} & s \in A \\ \text{unknown} & s \notin P \cup A \end{cases} \quad (1)$$

For each disease label d, let W_d be its weighted symptom set and R_d its required-premise set, with R_d contained in W_d. The academic implication is a conjunction of the propositions in R_d leading to the rule label d. In this article, fired means that the conjunction is satisfied by present tokens. It does not mean

that the patient has the corresponding disease. Several rules can fire for the same fact set.

$$\bigwedge_{s \in R_d} s \rightarrow d \tag{2}$$

IV. REASONING AND EXPLANATION METHOD

A. Weighted rule overlap

The forward evaluation routine visits each disease entry once. It computes matched weighted symptoms M_d, required premises not present U_d, and required premises explicitly absent B_d. An absent required premise is also not present and therefore appears in U_d. These sets control the score and displayed rule state.

$$M_d = \mathrm{P} \cap W_d, \qquad U_d = R_d \setminus \mathrm{P} \qquad B_d = R_d \cap \mathrm{A} \tag{3}$$

Let $w_{d,s}$ be the positive integer weight assigned to symptom s for rule d. The denominator T_d is the total weight of the rule's six symptoms. The implementation multiplies normalized matched weight by a fixed penalty when any required premise is not present. Only candidates with a nonempty M_d enter the ranked result list.

$$q_d = 100 \frac{\sum_{s \in M_d} w_{d,s}}{T_d} \lambda_d \qquad T_d = \sum_{s \in W_d} w_{d,s} \tag{4}$$

$$\lambda_d = \begin{cases} 1, & U_d = \varnothing \\ 0.60, & U_d \neq \varnothing \end{cases} \tag{5}$$

The score lies between 0 and 100 because weights are positive and matched weight cannot exceed total weight. Scores are sorted in descending order, with an alphabetical comparison of labels for equal values. The interface rounds the displayed value to one decimal place. It does not calibrate the score against prevalence, outcome labels, or likelihood ratios. A percentage in this interface is therefore a weighted coverage measure, not diagnostic confidence.

The penalty is applied once, regardless of how many required symptoms are missing. A single missing premise can produce a substantial discontinuity when its answer changes to Yes, since the system both adds its weight and removes the penalty. Scores are also normalized independently for each rule, so two labels with equal matched weight can receive different scores when their total weights differ. These properties follow directly from the formula rather than from clinical evidence.

B. Hypothesis-directed premise checks

After selecting a hypothesis, the user can answer each missing required premise with Yes, No, or Unsure. Yes adds the symptom to P and removes it from A. No removes it from P and adds it to A. Unsure removes it from both sets. Each answer invokes a fresh evaluation, allowing the ranked order to change while the selected candidate remains active if it still has symptom overlap.

The displayed status prioritizes explicit absence: a nonempty B_d yields blocked; otherwise, a nonempty U_d yields not fired; otherwise, the implication is fired. The implementation continues to rank a blocked candidate whenever it retains a matched symptom. An absent required premise has no further numerical penalty beyond the 0.60 missing-premise multiplier. Consequently, status and rank must be read together; a high position does not imply a satisfied rule.

C. Consistency gate and execution sequence

The declared incompatible pairs are fever with no_fever, high_fever with no_fever, and diarrhea with constipation. If both tokens of a pair occur in P, run clears the ranked state, displays the conflicting input, and returns before scoring. These are project-defined constraints rather than universal clinical incompatibilities. The checker does not derive additional contradictions from a logical theory and does not establish consistency of the entire knowledge base.

Algorithm 1 Implemented evaluation

```
Input: present facts P, absent facts A, dictionary KB
If a declared conflict pair is contained in P:
    clear results, display conflict, and return
For every rule d in KB:
    compute matched, missing, and absent premises
    compute normalized overlap and premise multiplier
    retain d if at least one weighted symptom matches
Sort retained rules by score, then by label
Display candidate states and the selected rule trace
On a premise answer, update P and A and repeat
```

D. Explanation and warning rendering

The result trace lists the conjunction, rule status, matched weight, total weight, missing-premise multiplier, and score. The knowledge-base view separately exposes rule weights and source flags. Treatment and precaution strings remain in the data structure, but the browser result renderer does not display them as treatment recommendations. Its output advises clinician assessment rather than generating a prescription.

A dynamic notice is generated when present facts intersect a list of seven warning tokens. This test is independent of candidate scores within result rendering. However, the rendering step follows the contradiction gate, so a conflicting input prevents that dynamic notice from being shown. An always-visible educational notice remains part of the page. An additional message reflects the selected rule's urgency flag. Neither mechanism is a validated triage algorithm.

E. Computational cost

Let D be the number of rules, L the number of weighted links, and R the number of required links. Assuming expected constant-time set membership, the scan costs O(L + R). Sorting costs O(D log D) in the worst case, excluding label-comparison cost. Working results store matched and required-premise lists, giving O(L + R + D) space. These are structural complexity estimates. No browser latency, memory benchmark, or scalability experiment was conducted.

V. SOFTWARE VERIFICATION AND RESULTS

A. Verification protocol

Verification executed the inline JavaScript from the supplied HTML in a Node.js VM context. A minimal document stub supplied the elements referenced during initialization and captured generated result HTML. The harness called the original evaluate and run routines, manipulated their existing fact sets, and invoked the registered premise-answer handler. It did not replace the scoring implementation. The checks were performed for this manuscript; they are not pre-existing experimental results reported in the PDF.

The protocol combined dictionary checks, constructed symptom profiles, and targeted control-flow checks. All 38 entries were inspected for six positive integer weights and valid required-premise references. Each rule's full weighted profile was supplied to evaluate, and every required premise was separately removed from that profile. Additional assertions examined declared conflicts, empty and unlisted inputs, demo arithmetic, answer transitions, tied scores, and warning HTML. Table II reports assertion counts. They are deterministic software checks, not a dataset of patient cases.

TABLE II EXECUTED CHARACTERIZATION ASSERTIONS

Assertion family	Checks	Matched
Rule structure	38	38
Complete profiles	38	38
Required-premise deletion	79	79
Conflict pairs	3	3
Empty, unlisted, and tie inputs	3	3
Example score arithmetic	2	2
Answer and blocked states	4	4
Individual warning tokens	7	7
Conflict before warning	1	1
Total	175	175

All 175 assertions agreed with the characterized behavior. Of these, 38 complete-profile checks verified a score of 100 with no missing or blocked premises. The 79 premise-deletion checks verified that each removed required symptom became missing and that the score followed the 0.60 formula. The three declared contradictory pairs each cleared results and produced conflict HTML. The four answer-state assertions verified No, Unsure, and Yes transitions and the continued ranking of a blocked candidate.

The seven warning-token assertions confirmed dynamic notice generation for individual warning inputs. A separate assertion confirmed the limitation that adding a contradiction to a warning input yields conflict HTML without the dynamic notice. Passing this assertion characterizes a control-flow limitation; it is not evidence that warning handling is clinically safe. Likewise, assertions on complete profiles establish arithmetic and state behavior, not disease-identification accuracy.

B. Worked example

The built-in example selects high_fever, body_pain, fatigue, and dry_cough. Influenza matches weights 10, 9, 8, and 7, giving 34 out of a total of 45. Its required premises are present, so its displayed score is 75.6. Dengue Fever matches high_fever and body_pain, giving 19 out of 46, with its required premises present and a displayed score of 41.3. Table III lists the first five returned candidates. These are illustrative rule outputs from constructed input.

TABLE III BUILT-IN EXAMPLE OUTPUT

Candidate	Score	Premises
Influenza	75.6	Met
Dengue Fever	41.3	Met
Viral Fever	20.8	Incomplete
COVID-19	16.5	Incomplete
Iron Deficiency Anemia	12.5	Incomplete

Viral Fever and COVID-19 remain incomplete because fever is a different token from high_fever. The system does not infer fever from high_fever. Iron Deficiency Anemia is incomplete because weakness is unknown. This example shows why vocabulary semantics matter: a label that appears clinically related to another label does not acquire an implication unless that relation is encoded.

A second constructed input uses only high_fever and body_pain. Influenza has matched weight 19 out of 45 and an unknown fatigue premise, producing 25.3 after the multiplier. Answering No for fatigue changes its status to blocked while preserving that score. Answering Unsure restores an unknown premise. Answering Yes gives matched weight 27 out of 45 and removes the multiplier, yielding 60.0. The sequence exposes the difference between numeric overlap and the logical state of a required premise.

C. Interpretation of the evidence

The verification supports the reported counts, formula implementation, candidate ordering, and selected interaction paths. It does not test full browser layout, keyboard navigation in a real browser, assistive technology behavior, or all possible symptom combinations. The direct full-profile and premise-deletion tests use evaluate rather than the complete run path; only the targeted conflict and warning tests exercise the associated rendering sequence. The document stub validates generated HTML strings rather than visual presentation.

No labeled patient cohort, clinician-adjudicated diagnosis, or external medical benchmark was used. Accuracy, sensitivity, specificity, precision, recall, and clinical safety rates are therefore not reported. There is also no measured comparison with another diagnostic system or a user study of learning outcomes. The results establish a bounded characterization of the implemented symbolic demonstration.

VI. DISCUSSION AND LIMITATIONS

A. Educational value and knowledge quality

The interface places evidence, missing premises, and the score calculation together, allowing a learner to inspect why a candidate appears. Interactive answers demonstrate how partial information changes both premise status and ordering. Explicit unknown states prevent an unchecked checkbox from automatically becoming a negative assertion. These features make the reasoning process inspectable, although their effect on student understanding remains to be evaluated.

Transparency exposes rather than resolves weaknesses in the knowledge base. Shared symptoms allow multiple implications to fire. Missing contextual variables include age, symptom duration, measurement thresholds, comorbidities, exposure history, and laboratory evidence. Required symptoms are project antecedents, not established necessary or sufficient conditions for the corresponding diseases. The fixed penalty has no documented calibration, and a rule's urgency flag does not establish urgency for a particular person.

B. Observed implementation constraints

Two verified behaviors deserve priority in future revisions. First, a blocked candidate remains in the same rankable population as other partial matches. A clearer interface could show blocked hypotheses in a separate state or exclude them from the main ranking, with the rule for doing so explicitly defined. Second, dynamic warning handling follows the contradiction gate. Computing and displaying warnings before that gate would make the warning path independent of consistency validation. These changes are proposed and were not made to the supplied application.

Vocabulary design also requires review. fever and high_fever have no declared entailment relationship, while no_fever and an absent answer to fever use different representations. A documented symptom ontology could normalize these states, but any hierarchy or incompatibility relation would require domain review. The diarrhea-constipation conflict illustrates the need to distinguish an interface simplification from a clinical rule that is valid in every temporal or contextual setting.

C. Validation required for broader claims

A stronger software study should extend verification to browser-level interaction, accessibility, and combinations that cross state and warning boundaries. It should also preserve versioned rules and test artifacts so that changes in outputs can be traced to changes in the knowledge base. An educational study would need a defined learner population, tasks, comparison condition, and measured outcomes before claiming improved learning.

Any future clinical study would need professionally reviewed rules, a defined intended use, an appropriately governed dataset, and independent reference labels. Evaluation should separate ranking quality, abstention behavior, and urgent-condition handling rather than treating overlap percentages as probabilities. Relevant oversight and participant protections would have to be established before such work. These are future requirements; no clinical enrollment, ethical approval, or external validation is claimed here.

VII. CONCLUSION

MEDIXAI demonstrates classical AI through a compact, inspectable combination of weighted rule overlap and required-premise checking. The implemented artifact contains 38 rules, 104 symptom identifiers, and eight categories. Its explanation interface distinguishes matched evidence, unknown premises, and explicitly absent premises while exposing the score calculation. A characterization harness executed 175 assertions and confirmed both expected arithmetic behavior and specific control-flow limitations.

The study supports use of the artifact as an educational demonstration of symbolic reasoning. It does not support diagnostic or triage claims. The next implementation priorities are clearer treatment of blocked candidates, warning handling independent of input consistency, and documented vocabulary semantics. Claims about educational benefit or medical performance require separate studies designed for those outcomes.

REFERENCES

[1] FOAI Project, expanded MEDIXAI implementation, pp. 7-35, unpublished project document, n.d. Supplied file: FOAI.pdf.

[2] MEDIXAI, browser-based academic expert-system prototype, unpublished HTML/JavaScript artifact, n.d. Supplied file: medixai.html.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021. [Online source]

[4] E. H. Shortliffe and B. G. Buchanan, "A model of inexact reasoning in medicine", *Mathematical Biosciences*, vol. 23, nos. 3-4, pp. 351-379, 1975, doi: 10.1016/0025-5564(75)90047-4. [Online source]

[5] B. G. Buchanan and E. H. Shortliffe, Eds., *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Reading, MA, USA: Addison-Wesley, 1984. [Online source]

[6] J. A. Robinson, "A machine-oriented logic based on the resolution principle", *Journal of the ACM*, vol. 12, no. 1, pp. 23-41, Jan. 1965, doi: 10.1145/321250.321253. [Online source]

[7] World Health Organization, *Ethics and Governance of Artificial Intelligence for Health: WHO Guidance*. Geneva, Switzerland: WHO, 2021. ISBN: 9789240029200. [Online source]